

Software Engineering 2 – Final Group Presentation Rubric

Overview

The presentation is scored out of **100 points** across 11 criteria in three categories, then adjusted per student by a peer-evaluation contribution factor.

Category	Criteria	Points
A. Technical content	Problem and objectives, requirements and scope, design and architecture, implementation, testing and QA, project management	60
B. Live demonstration	Working system demo	15
C. Delivery	Organization and time, visual aids, speaking, Q&A	25
Total		100

Suggested format: 20 minutes presentation and demo, 10 minutes Q&A. Every member must present a part and answer at least one question.

Performance levels: Excellent (about 90–100% of the criterion's points), Proficient (75–89%), Developing (55–74%), Beginning (below 55%).

Rubric

A. Technical content (60 points)

Criterion	Excellent	Proficient	Developing	Beginning
1. Problem and objectives (10)	9–10: Problem, target users and measurable objectives are clearly stated and well justified.	7–8: Problem and users are clear; objectives are stated but not all measurable.	5–6: Problem is described vaguely; objectives are unclear.	0–4: No clear problem statement or objectives.
2. Requirements and scope (10)	9–10: Functional and non-functional requirements are complete, prioritized and traceable to features shown.	7–8: Key requirements are covered with minor gaps; scope is clear.	5–6: Requirements are incomplete or loosely tied to the system.	0–4: Requirements are missing or not presented.
3. System design and architecture (15)	14–15: Architecture, data model and key UML diagrams are accurate and readable; design decisions and trade-offs are explained.	11–13: Diagrams are mostly accurate; most decisions are explained.	8–10: Diagrams are incomplete or inconsistent with the system; little rationale.	0–7: Design is missing, incorrect or unexplained.

Criterion	Excellent	Proficient	Developing	Beginning
4. Implementation and technical decisions (10)	9–10: Tech stack, design patterns and code structure are justified; challenges and solutions are discussed in depth.	7–8: Implementation is explained with reasonable justification.	5–6: Implementation is described superficially; choices are not justified.	0–4: Little or no explanation of how the system was built.
5. Testing and quality assurance (10)	9–10: Test strategy (unit, integration, user acceptance) is shown with evidence such as results, coverage or bug tracking.	7–8: Testing was done and shown, with some gaps in evidence.	5–6: Testing is mentioned with little evidence.	0–4: No testing presented.
6. Project management and process (5)	5: Methodology (e.g. Scrum), timeline, task allocation and version control use are shown with lessons learned.	4: Process is described with some evidence.	3: Process is mentioned briefly.	0–2: No evidence of a managed process.

B. Live demonstration (15 points)

Criterion	Excellent	Proficient	Developing	Beginning
7. Working system demo (15)	14–15: Core features run smoothly end to end; demo follows realistic user scenarios tied to the requirements.	11–13: Most features work; minor glitches are handled well.	8–10: Several features fail or are incomplete; demo is disorganized.	0–7: System does not run, or demo is replaced by screenshots only.

C. Delivery (25 points)

Criterion	Excellent	Proficient	Developing	Beginning
8. Organization and time management (5)	5: Logical flow, smooth transitions, finishes within the time limit.	4: Mostly logical; within about 2 minutes of the limit.	3: Some disorganized parts; noticeably over or under time.	0–2: Disorganized; far over or under time.
9. Visual aids (5)	5: Slides are clean, readable and support the talk; diagrams are legible.	4: Slides are clear with minor issues.	3: Slides are text-heavy or hard to read.	0–2: Slides are missing or distract from the talk.

Criterion	Excellent	Proficient	Developing	Beginning
10. Speaking and teamwork in delivery (5)	5: All members speak confidently and clearly, with eye contact; parts are balanced.	4: Members speak clearly; parts are slightly unbalanced.	3: Some members read from slides or are hard to follow.	0–2: One or two members carry the presentation.
11. Q&A (10)	9–10: Answers are accurate and confident; any member can answer about any part of the system.	7–8: Most answers are correct; a few members defer to others.	5–6: Answers are vague or partly incorrect.	0–4: Group cannot answer basic questions about its own system.

Individual contribution

Each student's grade = group score × contribution factor. The factor comes from a confidential peer evaluation, checked against your own observation and the repository history.

Each student rates every teammate (not themselves) from 1 to 4 on these four items:

Item	4 – Excellent	3 – Good	2 – Fair	1 – Poor
Technical contribution	Major share of code, design or testing	Fair share of the work	Less than a fair share	Little or no work
Reliability	Always met deadlines	Usually met deadlines	Often late	Rarely delivered
Communication and collaboration	Active in meetings, helped others	Participated regularly	Participated occasionally	Unresponsive
Presentation preparation	Fully prepared, knew the whole system	Prepared own part well	Partly prepared	Unprepared

Average peer rating (out of 16)	Contribution factor
11–16	1.00
8–10	0.85
5–7	0.70
4 or below	0.50

No bonus: the highest factor is 1.00, so no student scores above the group score.

Group score	Factor	Calculation	Individual grade
90	1.00	90 × 1.00 = 90	90
90	0.85	90 × 0.85 = 76.5	76.5
90	0.70	90 × 0.70 = 63	63

You may override the factor when peer ratings conflict with commit history or what you saw during the term.

Score sheet

Group name: _____ Project title: _____ Date: _____

Criterion	Max	Score	Comments
1. Problem and objectives	10		
2. Requirements and scope	10		
3. System design and architecture	15		
4. Implementation and technical decisions	10		
5. Testing and quality assurance	10		
6. Project management and process	5		
7. Working system demo	15		
8. Organization and time management	5		
9. Visual aids	5		
10. Speaking and teamwork in delivery	5		
11. Q&A	10		
Group total	100		

Member	Contribution factor	Individual grade

Overall feedback:

Strengths:

Areas for improvement:

Evaluator signature: _____

Q&A question bank

Pick one or two questions per member, mixing the two units, and score answers under criterion 11. Each question asks students to apply the lesson to their own project; the right column lists what a strong answer includes.

Unit 1: Managing changes

Configuration management

Question	A strong answer includes
What are the configuration items in your project, and how did you identify them?	Source code, requirements, design documents, test cases, build scripts, config files, database scripts; items are named and versioned.
Walk us through how a change request was handled in your project, from request to approval to implementation.	Request logged, impact assessed, approved by a person or role, implemented, tested, recorded; a real example from the project.
What is a baseline, and at what points did your team set one?	Baseline = an approved, fixed version used as a reference; examples like end of requirements, end of each sprint, release.
How would your team handle two members needing different versions of a library or environment?	Dependency files (e.g. package.json, requirements.txt), lock files, environment configuration, containers or documented setup.

Version control

Question	A strong answer includes
Describe your branching strategy and why you chose it.	Names a strategy (feature branches, Git Flow, trunk-based); explains main vs. development vs. feature branches and why it suited the team.
Tell us about a merge conflict you had. How did you resolve it, and how could it have been avoided?	Concrete example; how the conflict was read and fixed; prevention through smaller commits, frequent pulls, clear task ownership.
How did your team review code before merging into the main branch?	Pull requests, reviewers, review checklist, tests run before merge, branch protection.
Show us your commit history. What makes a good commit message, and do yours follow it?	Small focused commits; short summary of what and why; honest assessment of their own history.

Release management

Question	A strong answer includes
How do you number your releases, and what does each part of the version number mean?	Semantic versioning (major.minor.patch) or a stated scheme; when each part increases.
What steps does your team follow before releasing a new version?	Freeze features, run full tests, update documentation and release notes, tag the release, build, deploy, verify.
If a release introduced a critical bug in production, how would you roll back?	Tagged previous version, redeploy it, database migration rollback or backup, then hotfix branch.
What goes into your release notes, and who are they written for?	New features, fixes, known issues, installation or upgrade steps; written for users or administrators.

Unit 2: Software testing

Testing introduction

Question	A strong answer includes
What levels of testing did you perform, and give	Unit, integration, system, acceptance; a specific

Question	A strong answer includes
one example of each from your system?	test for each.
Explain the difference between black-box and white-box testing using a feature of your system.	Black-box tests inputs and outputs from the spec; white-box tests internal paths and logic; same feature tested both ways.
Why can't testing prove that your software has no defects?	Testing shows the presence of defects, not their absence; exhaustive testing is impossible; risk-based selection.
Which part of your system did you consider highest-risk, and how did that affect your testing?	Names a module (e.g. payments, login, data saving); more test cases, edge cases or earlier testing there.

Verification and validation

Question	A strong answer includes
What is the difference between verification and validation? Give an example of each from your project.	Verification: are we building the product right (meets spec, reviews, inspections). Validation: are we building the right product (meets user needs, user testing).
How did you confirm that the system actually meets your users' needs, not just your requirements document?	User acceptance testing, demos with real or representative users, feedback collected and acted on.
What static verification activities did your team do, such as reviews or inspections?	Code reviews, requirements or design walkthroughs, linting or static analysis tools; what they found.
Pick one requirement and show how you traced it to a test case and its result.	Requirement ID → test case ID → pass/fail result; mentions a traceability matrix.

Testing documentation

Question	A strong answer includes
What testing documents did your team produce?	Test plan, test cases or scripts, test data, test results or summary report, defect log.
Show us one test case. What fields does it contain, and why does each matter?	ID, description, preconditions, steps, test data, expected result, actual result, status.
What is in your test plan, and did the team follow it?	Scope, approach, schedule, resources, entry and exit criteria, risks; honest comparison of plan vs. reality.
How did you record and track bugs found during testing?	Bug tracker or issue log; severity, priority, steps to reproduce, status, assigned person; retesting after fixes.

Software maintenance

Question	A strong answer includes
Name the four types of software maintenance and give an example of each that could apply to your system.	Corrective (fix bugs), adaptive (new OS, browser or API), perfective (new features, performance), preventive (refactoring, updating dependencies).
If a new team took over your project tomorrow, what would make it easy or hard for them to	Documentation, README, code readability, comments, tests, consistent structure; names

Question	A strong answer includes
maintain it?	actual weak spots.
What is regression testing, and how would you do it when changing your system later?	Retesting existing features after a change; automated test suite, re-running key test cases.
What technical debt does your project have, and how would you address it?	Honest list (hard-coded values, missing tests, duplicated code); plan to refactor and prioritize.